

Lecture 10: Reinforcement Learning
Value Functions and Bellman Equations
Q-Learning and Policy Gradients
Deep Networks in Place of Tables

Deep Learning for Scientific Discovery



Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

Where We Are

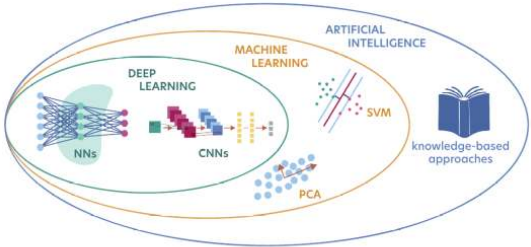
- Lectures 3–4: networks learn a map from inputs to labels, $\mathbf{x} \mapsto y$, from labelled examples (MNIST, blood cells, Ising configurations)
- Lectures 5–6: without labels, networks compress data and generate new samples (autoencoders, VAEs)
- Lecture 7: data ordered in time, where a model predicts the next state of a system from its past, but cannot change it
- Lecture 9: tables of measured quantities, about which decision trees and random forests ask readable questions
- **Today**: an agent **acts**. Its actions change the state of its environment, and the only feedback is a reward, often delayed. There are no labels telling it which action was right.

1

Learning from Rewards

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Programming, Machine Learning, Reinforcement Learning



Main ML paradigms:

1. Unsupervised Learning
2. Supervised Learning

Teacher-Student scheme

3. Reinforcement Learning

Finding best strategy





Go and AlphaGo

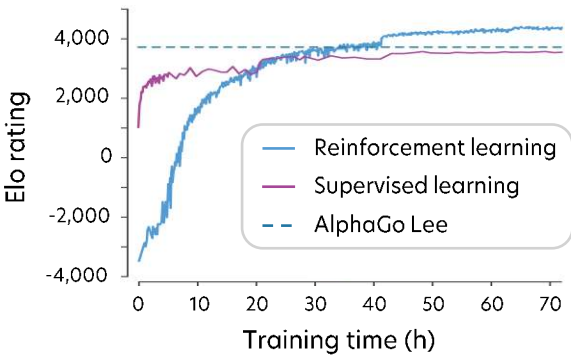


Seoul, March 2016

- Go is played on a board of 19×19 points and has about 10^{170} legal positions, far more than the about 10^{80} atoms in the observable universe
- a move is good or bad only in view of the end of the game, often hundreds of moves later, and for decades the game was considered out of reach of computers
- AlphaGo beat the European champion Fan Hui 5:0 in October 2015 and Lee Sedol 4:1 in March 2016
- it first learned from expert games (supervised) and then improved by playing against itself

D. Silver *et al.*, Mastering the game of Go with deep neural networks and tree search, Nature **529**, 484 (2016)

AlphaGo Zero Learns Without Human Games



The **Elo rating** measures playing strength from game results. A strong player who loses to a newcomer loses many points, an expected result changes little, and 400 points more wins about 91% of games.

blue AlphaGo Zero, self-play only, given only the rules
purple the same network trained on expert games (supervised)
dashed, the version of AlphaGo that beat Lee Sedol

The supervised start gives an early lead and then limits the player. Pure reinforcement learning overtakes it and passes the Lee Sedol version after about 36 hours.

D. Silver *et al.*, Mastering the game of Go without human knowledge, Nature **550**, 354 (2017)

Robots Trained with Reinforcement Learning

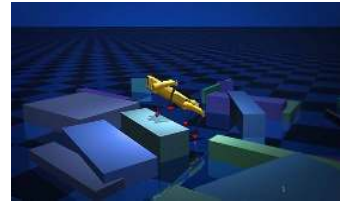
Recent robots of Boston Dynamics use locomotion policies learned with reinforcement learning in simulation. Click a picture to open the video.



Walk, Run, Crawl, RL Fun (2025), Atlas with policies learned with RL from motion capture



Atlas Airborne (2026), full-body control, acrobatics and failed attempts



Reinforcement learning with Spot, a policy trained in a million simulations

Boston Dynamics, with the RAI Institute, Boston Dynamics blog on reinforcement learning with Spot

Hide and Seek

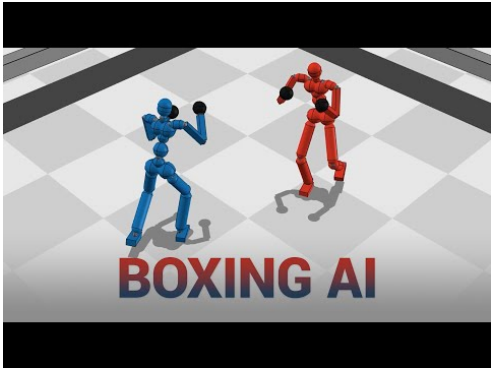


click to open the video (Two Minute Papers)

- hiders and seekers play in a simulated room with movable boxes and ramps
- the reward refers only to the game, $+1$ for the hiders if all are hidden and -1 if any is seen, the opposite for the seekers
- nothing in the reward mentions boxes, ramps or tools
- the teams play each other for hundreds of millions of games
- strategies appear one after another, chasing, shelters built from boxes, ramps over walls, locked ramps, "box surfing"
- every new strategy of one team is a harder task for the other

B. Baker *et al.*, Emergent tool use from multi-agent autocurricula, ICLR 2020, arXiv:1909.07528, OpenAI, blog post with videos

Two Boxers Learning Against Each Other



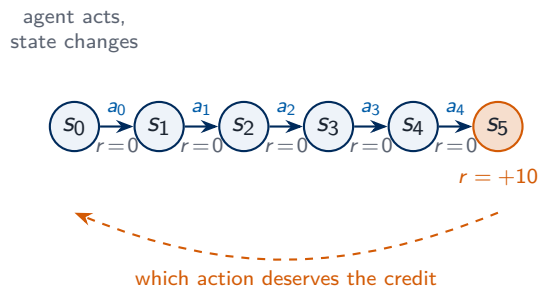
click to open the video (Two Minute Papers)

- two physically simulated humanoids, every joint driven by forces, fight in a boxing ring
- they first learn basic movements from a short recording of a human boxer
- they then learn to fight by reinforcement learning against each other, with a reward for hitting the opponent and a penalty for being hit
- tactics appear that nobody programmed, keeping distance, jabs, dodging, counter-punches
- the same method trains two fencers

J. Won, D. Gopinath and J. Hodgins, Control strategies for physically simulated characters performing two-player competitive sports, ACM Trans. Graph. **40**, 4 (2021), Facebook AI Research, project page

No Teacher, Only Rewards

- there are no labelled data and no loss function given in advance
- the agent creates its data by acting, so a poor strategy sees only poor situations
- rewards can come many steps after the action that caused them



On which quantity should we focus, what plays the role of the loss function, and how do we optimise it?

Two Ways to Learn Behaviour from Rewards

learn values

how good is each situation and each action act on the best value

Bellman equations → Q-learning

learn the policy

change the probabilities of the actions directly, towards what worked

policy gradient (REINFORCE) with a baseline

both start from the same dictionary of state, action, reward, policy and return

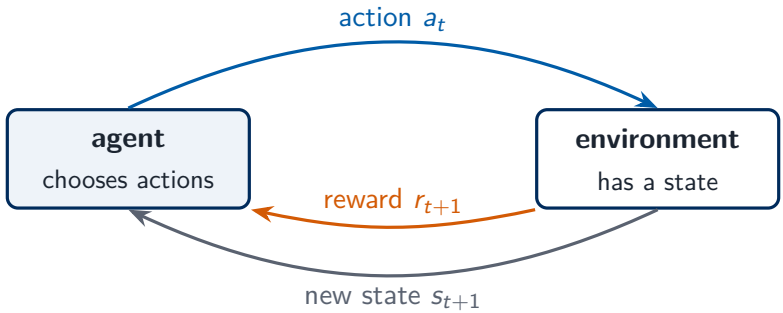
We first build the value route, from value functions through the Bellman equations to Q-learning, and then the policy route. The two meet in the baseline of the policy gradient, which is a value function. At the end both tables, of values and of action probabilities, are replaced by neural networks, which is how the Lunar Lander and the cart-pole are controlled.

2

The Markov Decision Process

JAGIELLONIAN UNIVERSITY
IN KRAKOW

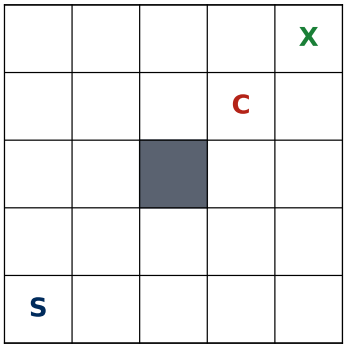
Agent and Environment



- the agent observes the state s_t and chooses an action a_t
- the action changes the state of the environment, $s_t \to s_{t+1}$
- the environment returns a reward r_{t+1}, a single number

time runs in discrete steps $t = 0, 1, 2, \ldots$

The Rover Gridworld



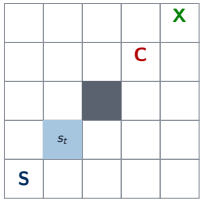
S landing site, X sample site, C crater, grey boulder

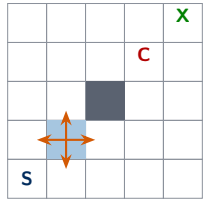
Agent and task

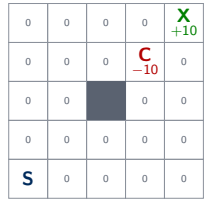
The agent is a rover on Mars, on a terrain of 5×5 tiles. It must drive from the landing site S to the sample site X in as few steps as possible and avoid the crater C. It knows neither the map nor the slip probabilities and sees only its tile and the reward after each step. What it must learn is a policy, one direction for every tile.

$\mathcal{S}$	the tiles (row, column), 24 states
$\mathcal{A}$	$\uparrow \rightarrow \downarrow \leftarrow$ on every tile
$p(s' \mid s, a)$	intended direction 0.8, each perpendicular direction 0.1 (slippery sand), and the boulder or the edge keeps the rover on its tile
r	$+10$ for entering X, -10 for entering C, 0 otherwise
γ	0.9, so a shorter path is worth more

State, Action and Reward







state space $\mathcal{S}$, the set of all states of the environment, here the 24 tiles on which the rover can stand

also a board position in Go, or position, velocity and angle of a lander

action space $\mathcal{A}$, the set of all actions of the agent, here a step $\uparrow \rightarrow \downarrow \leftarrow$ on every tile

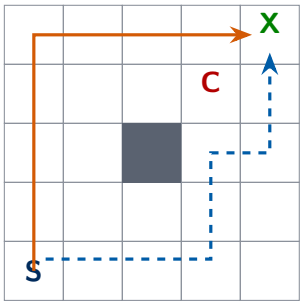
also a move in Go, or firing one engine of a lander

reward $r_{t+1} \in \mathbb{R}$, one number after every action, here $+10$ for X, -10 for C and 0 otherwise

the episode ends at X or C

The reward says what is good now. It does not say which action was right.

The Return



two paths from S to X, and next to the crater a slip can end the episode with -10

Return

$$R_{\text{tot}} := r_1 + r_2 + r_3 + \cdots + r_T$$

R_{tot} is the undiscounted total reward and T the final time step of the episode. The agent chooses its actions so that the return is as large as possible on average over the slips.

A step without reward can pay off later, and a short path can end in the crater, so maximising the next reward alone is not enough.

Discount Factor and the Recursion of the Return

Discounted return from time t

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k \, r_{t+k+1} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots, \qquad \gamma \in [0, 1]$$

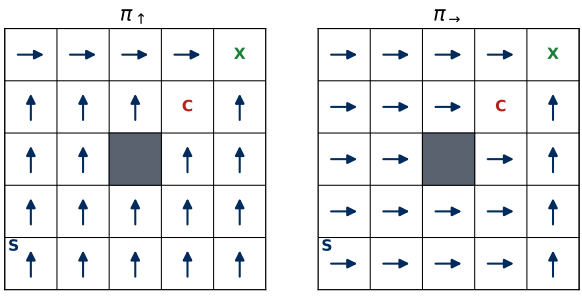
- a reward k steps ahead counts with weight γ^k, so the agent looks about $1/(1-\gamma)$ steps ahead, 10 steps for $\gamma = 0.9$ and 100 for $\gamma = 0.99$
- for $\gamma \to 0$ the agent settles for early small rewards, for $\gamma \to 1$ it accepts poor early steps for a larger later reward

Taking the first reward out of the sum gives the recursion

$$G_t = r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \qquad \Longrightarrow \qquad \boxed{G_t = r_{t+1} + \gamma \, G_{t+1}}$$

rewards $0, 0, 0, 10$ and $\gamma = 0.9$ give $G_3 = 10$, $G_2 = 9$, $G_1 = 8.1$, $G_0 = 7.29$

The Policy



two deterministic policies, one arrow per tile

Policy

$$\pi(a \mid s) \coloneqq p(a_t = a \mid s_t = s)$$

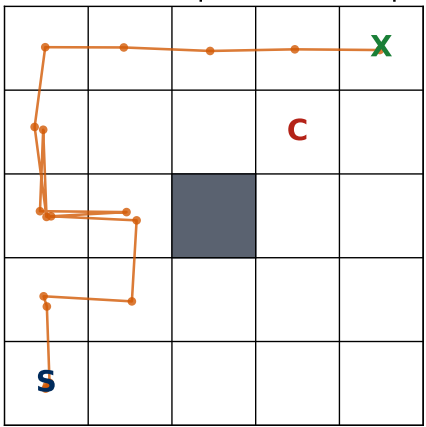
the probability of choosing action a in state s, the strategy of the agent

A deterministic policy gives one action per state, and the random policy gives every action probability $1/4$.

Learning means finding a policy with a large expected return.

One Episode of a Random Agent

one random episode, 17 steps



With $\pi(a \mid s) = 1/4$ the agent records $(s_t, a_t, r_{t+1}, s_{t+1})$ at every step.

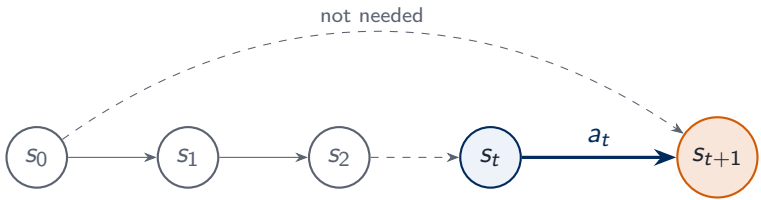
t	s_t	a_t	r_{t+1}	s_{t+1}
0	(4,0)	$\leftarrow$	+0	(4,0)
1	(4,0)	$\rightarrow$	+0	(4,0)
2	(4,0)	$\uparrow$	+0	(3,0)
3	(3,0)	$\downarrow$	+0	(3,0)
⋮			⋮	
15	(0,2)	$\downarrow$	+0	(0,3)
16	(0,3)	$\rightarrow$	+10	X

Only the last reward, after 17 steps, is nonzero. The plain sum gives $R_{\text{tot}} = 10$, as much as a direct path of 8 steps. With $\gamma = 0.9$ the episode is worth $10 \cdot 0.9^{17-1}$ and the direct path $10 \cdot 0.9^{7} \approx 4.8$, so the discount prefers the faster agent.

Markov Property

Markov property

$$p(s_{t+1} \mid s_0, a_0, \ldots, s_t, a_t) = p(s_{t+1} \mid s_t, a_t)$$



Given the present state and action, the future is independent of the past. The state contains everything relevant for the future evolution, as a chess position, the position and velocity of a lander, or a tile of the terrain do. If the environment has memory, we restore the Markov property by extending the state with the remembered information.

Dynamics of the Environment

Environment dynamics

$$p(s', r \mid s, a) \coloneqq p(s_{t+1} = s',\ r_{t+1} = r \mid s_t = s,\ a_t = a)$$

where the agent lands after taking a in s, and which reward it gets. This table is the full description of the environment, its *model*.
On the tile below X the action ↑ gives $(\mathrm{X}, +10)$ with probability 0.8, a slip into the crater $(\mathrm{C}, -10)$ with 0.1 and a slip against the edge $(s, 0)$ with 0.1.

state-transition probability

$$p(s' \mid s, a) \coloneqq \sum_r p(s', r \mid s, a)$$

the reward is summed out

expected reward

$$r(s, a) \coloneqq \sum_r \sum_{s' \in \mathcal{S}} r\, p(s', r \mid s, a)$$

below X with ↑,
$0.8 \cdot 10 + 0.1 \cdot (-10) + 0.1 \cdot 0 = 7$

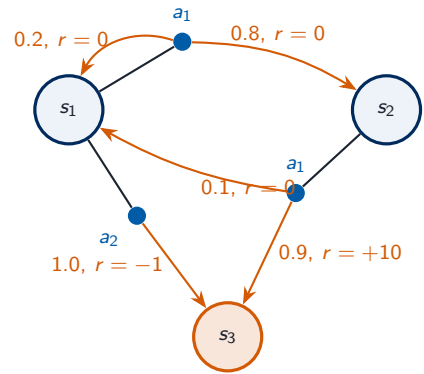
Markov Decision Process

A **Markov decision process** is the tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$ of

- the states $\mathcal{S}$ the environment can be in
- the actions $\mathcal{A}$ the agent can take
- the dynamics $p(s', r \mid s, a)$
- the discount γ, how much the future counts

The agent looks for a policy that maximises the expected return. The word *decision* says that the action of the agent changes the probabilities of what comes next.

The dynamics p always exist, but the agent usually does not know them and only sees samples (s, a, r, s').



circles are states, dots are actions chosen by the agent,

arrows carry $p(s', r \mid s, a)$, probability and reward

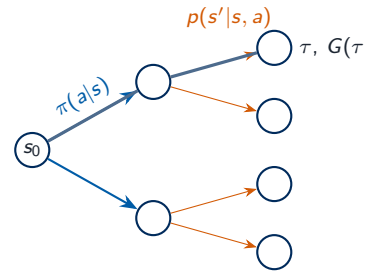
Probability of a Trajectory

A trajectory $\tau = (a_t, s_{t+1}, a_{t+1}, \ldots, s_T)$ from $s_t = s$ is produced alternately by the agent and by the environment,

$$p_\pi(\tau \mid s_t = s) \coloneqq \prod_{k=t}^{T-1} \underbrace{\pi(a_k \mid s_k)}_{\text{agent}} \underbrace{p(s_{k+1} \mid s_k, a_k)}_{\text{environment}}$$

and every trajectory collects its own discounted return $G_t(\tau)$.

The policy controls one factor of each step, the environment the other.

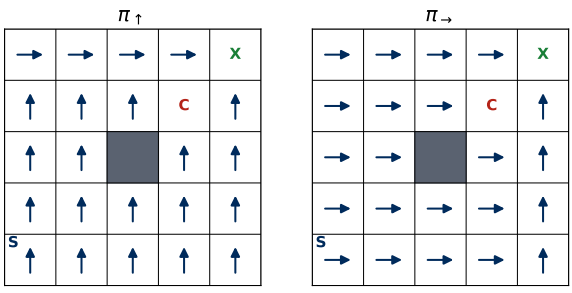


agent $\pi(a|s)$, environment $p(s'|s,a)$

3
Value Functions

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Two Policies to Compare



$\pi_\uparrow$ drives up to the top row and then right

$\pi_\rightarrow$ drives right to the last column and then up, past the crater

Which policy is better?

The answer depends on the tile where the rover starts, so one number per policy is not enough.

The **value function** tells, for every state, how good a policy is, and the **Bellman equations** compute it.

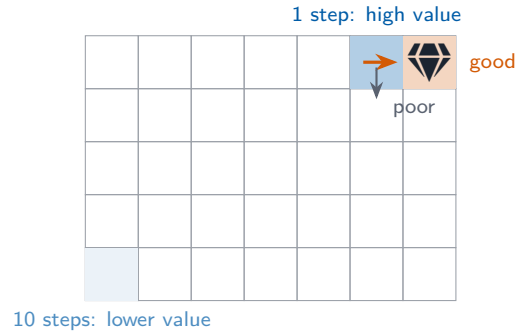
Value of a State

For someone looking for a treasure on a map, being one step away is better than being ten steps away.

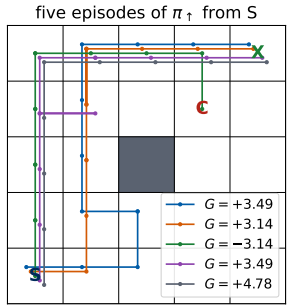
Near the treasure not all actions are equally good, since one leads to it and the others away from it.

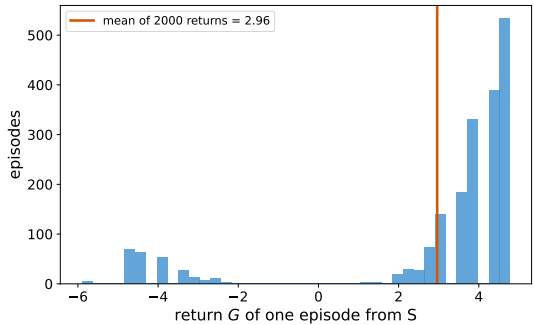
Each state s and each action a has a *value*, the return the agent can expect from there.

The future rewards depend on what the agent does next, so **values are defined for a given policy π.**



Many Trajectories, One Value





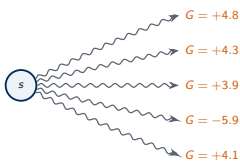
Gridworld, policy $\pi_\uparrow$ (up, then right along the top row), start S. Every episode takes a different path because of the slips, and collects its own return G. The value of S is the mean over episodes, and from 2000 episodes $V_{\pi_\uparrow}(S) \approx 2.96 \pm 0.06$.

State-Value Function

State-value function

$$V_\pi(s) := \mathbb{E}\big[G_t \mid s_t = s, \pi\big] = \sum_\tau p_\pi(\tau \mid s_t = s)\, G_t(\tau)$$

$$p_\pi(\tau \mid s_t = s) = \prod_{k=t}^{T-1} \pi(a_k \mid s_k)\, p(s_{k+1} \mid s_k, a_k), \qquad G_t(\tau) = \sum_{k=0}^{T-t-1} \gamma^k\, r_{t+k+1}$$



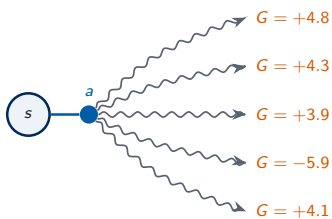
from s the agent follows π, and every trajectory has its own return G

how good it is to be in s, if the agent then follows π

- $\tau = (a_t, s_{t+1}, \ldots, s_T)$: a trajectory from $s_t = s$
- $p_\pi(\tau \mid s_t = s)$: its probability, a product of the agent's choices π and the environment's responses p
- $G_t(\tau)$: the discounted return collected along it, which gives one number $V_\pi(s)$ per state

gridworld, $\pi_\uparrow$, start S, $V_{\pi_\uparrow}(S) = 2.9$ (exact, from the Bellman equation)

Action-Value Function



the first action is fixed to a, afterwards the agent follows π

Action-value function

$$Q_\pi(s,a) = \mathbb{E}\big[G_t \mid s_t = s,\ a_t = a, \pi\big]$$

how good it is to do a now, and follow π afterwards

one number per state *and* action, which compares the actions available in s and lets the agent choose

gridworld, policy $\pi_\uparrow$, tile below X

$Q_{\pi_\uparrow}(s, \uparrow) = 7.69$, towards X

$Q_{\pi_\uparrow}(s, \leftarrow) = -6.53$, into the crater with probability 0.8

Advantage

$V_\pi(s)$ says how good the state is when the agent does what the policy usually does. $Q_\pi(s,a)$ says how good it is to take the particular action a first. Their difference removes how good the state itself is and leaves the effect of the choice.

Advantage

$$A_\pi(s,a) := Q_\pi(s,a) - V_\pi(s)$$

For $A > 0$ the action a is better than the usual behaviour of the policy in s, for $A < 0$ it is worse.

gridworld, $\pi_\uparrow$, tile below X, $V_{\pi_\uparrow} = 7.69$

$A(s,\uparrow) = 0.0$ for the action of the policy
$A(s,\leftarrow) = -14.22$, much worse than the policy

use

The advantage compares actions on a common scale, independent of how good the state is. Improving a policy means switching to actions with $A > 0$, and the baseline of the policy gradient builds on the same difference.

A. Dawid *et al.*, arXiv:2204.04198, chapter on reinforcement learning

4

Bellman Equations



JAGIELLONIAN UNIVERSITY
IN KRAKÓW

What We Are After

The value $V_\pi(s)$ is an average over all trajectories from s. Estimating it by playing episodes is slow and noisy (2000 episodes give one tile with an error of ± 0.06), and it has to be repeated for every tile and every policy.

Splitting the return into the first reward and the rest, $G_t = r_{t+1} + \gamma G_{t+1}$, links the value of a state to the values of its neighbours. The result is the Bellman equation

$$V_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \, V_\pi(s') \right],$$

one linear equation per state. With the best action in place of the average over actions it becomes the Bellman optimality equation.

- with a known model its solution gives the exact values of a policy without playing any episode
- the optimal policy picks in every state the action with the largest optimal action value
- with an unknown model the same equation is learned from samples (temporal differences, Q-learning)

A Chain Without Chance

The policy always moves right, $\gamma = 0.9$, and the reward is $+10$ on entering X and 0 otherwise.



From the definition, the value is the discounted sum of the rewards that follow,

$$V(s_3) = 10, \qquad V(s_2) = 0 + 0.9 \cdot 10 = 9, \qquad V(s_1) = 0 + 0.9 \cdot 0 + 0.9^2 \cdot 10 = 8.1.$$

The same numbers follow backwards from the end,

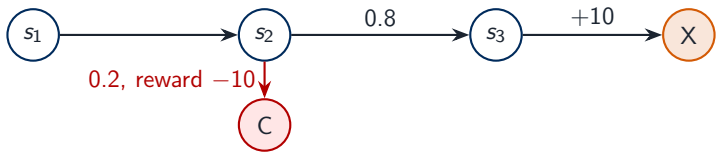
$$V(s_1) = 0 + 0.9 \cdot V(s_2), \qquad V(s_2) = 0 + 0.9 \cdot V(s_3).$$

value of a state = reward of the next step + $\gamma \times$ value of the next state

No episode has to be played to the end, because each value follows from its neighbour.

Adding Chance

In the same chain the sand at s_2 is slippery, and the rover slips into a crater C (reward -10, the episode ends) with probability 0.2.



At s_2 the next step is random, so going backwards from the end we average over its outcomes,

$$V(s_3) = 10, \quad V(s_2) = 0.8\,(0 + 0.9 \cdot 10) + 0.2\,(-10) = 5.2, \quad V(s_1) = 0.9 \cdot 5.2 = 4.68.$$

value = average over the next step of (reward + $\gamma \times$ value of the next state)

Without the slip $V(s_2) = 9$. A random policy adds a second average, over the actions.

Recursion of the State Value

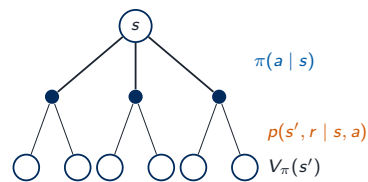
We start from the definition and use the recursion of the return, $G_t = r_{t+1} + \gamma G_{t+1}$,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}\big[G_t \mid s_t = s, \pi\big] \\ &= \mathbb{E}\big[\, r_{t+1} + \gamma G_{t+1} \mid s_t = s, \pi\big]. \end{aligned}$$

The value of s is the average of the first reward plus the discounted return from the next state.

Two random things happen in the first step.

- the agent chooses a with probability $\pi(a \mid s)$
- the environment answers with s' and r with probability $p(s', r \mid s, a)$



Bellman Equation for the State Value

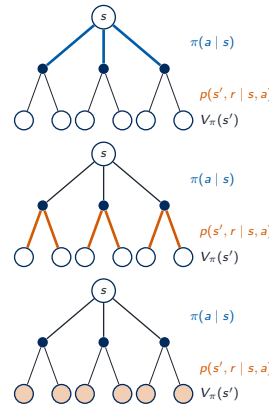
Averaging over the first action and over the answer of the environment (law of total expectation) gives

$$V_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \times \Big(r + \gamma \, \mathbb{E}\big[G_{t+1} \mid s_{t+1} = s', \pi \big] \Big).$$

The sums weight each action and each outcome by its probability. The Markov property and stationarity (p and π do not depend on t) give $\mathbb{E}[G_{t+1} \mid s_{t+1} = s', \pi] = V_\pi(s')$.

Bellman equation for the state value

$$V_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \big(r + \gamma \, V_\pi(s') \big)$$



Bellman Equation for the Action Value

The same steps with the first action fixed give

$$Q_\pi(s,a) = \mathbb{E}\big[G_t \mid s_t = s, a_t = a, \pi\big] = \mathbb{E}\big[r_{t+1} + \gamma G_{t+1} \mid s_t = s, a_t = a, \pi\big]$$
$$= \sum_{s',r} p(s', r \mid s, a)(r + \gamma\, V_\pi(s')).$$

Averaging $Q_\pi(s,a)$ over the actions of the policy reproduces the equation for V_π.

Relation between the two value functions

$$V_\pi(s) = \sum_a \pi(a \mid s)\, Q_\pi(s,a)$$

The value of a state is the average of the action values, weighted by the policy.

Solving the Bellman Equation with a Known Model

The **model** of the environment is the table $p(s', r \mid s, a)$ of the probabilities of every next state and reward. For the gridworld we wrote it ourselves (slip 0.8/0.1/0.1, rewards of X and C), whereas for a lander or a tokamak nobody gives it to us.

With a known model the Bellman equations are a linear system. With $P^{\pi}_{ss'} = \sum_a \pi(a \mid s)\, p(s' \mid s, a)$ and $r^{\pi}_s = \sum_a \pi(a \mid s)\, r(s, a)$,

$$V_\pi = r^\pi + \gamma P^\pi V_\pi \qquad \Longrightarrow \qquad V_\pi = \left(I - \gamma P^\pi\right)^{-1} r^\pi .$$

The matrix $I - \gamma P^\pi$, with I the identity, is invertible for $\gamma < 1$, since the eigenvalues of the stochastic matrix P^π have modulus at most 1.

chain, 3 equations with the solution 10, 5.2, 4.68

gridworld, 24 equations, $V_{\pi_\uparrow}(S) = 2.9$ from the linear system and 2.96 ± 0.06 from 2000 episodes

Value Maps of Three Policies

$V_{\pi_\uparrow}$

4.9	5.6	6.5	7.7	X
4.3	4.8	4.1	C	7.7
3.8	4.2		-8.3	5.3
3.4	3.2	-0.8	-5.7	3.6
S 2.9	2.5	-0.7	-3.9	2.5

$V_{\pi_\rightarrow}$

2.5	3.6	5.3	7.7	X
-3.9	-5.7	-8.3	C	7.7
-0.7	-0.8		4.1	6.5
2.5	3.2	4.2	4.8	5.6
S 2.9	3.4	3.8	4.3	4.9

$V_{\pi_{\text{rand}}}$

-0.9	-1.2	-1.7	-0.5	X
-0.9	-1.6	-4.2	C	-0.5
-0.6	-0.8		-4.2	-1.7
-0.5	-0.6	-0.8	-1.6	-1.2
S -0.4	-0.5	-0.6	-0.9	-0.9

The Bellman equation answers the question of the two policies. Blue marks a positive value and red a negative one. $\pi_\uparrow$ is good on the left and poor below the crater, and $\pi_\rightarrow$ is its mirror image. By symmetry both have $V(S) = 2.9$. The random policy has negative values almost everywhere, $V(S) = -0.37$.

Comparing Policies

$V_{\pi_\uparrow} - V_{\pi_\rightarrow}$

2.4	2.0	1.2	0.0	X
8.3	10.5	12.4	C	0.0
4.5	5.0		-12.4	-1.2
0.8	-0.0	-5.0	-10.5	-2.0
S -0.0	-0.8	-4.5	-8.3	-2.4

The value functions order the policies.

Partial ordering

$$\pi \geq \pi' \iff V_\pi(s) \geq V_{\pi'}(s) \text{ for all } s \in \mathcal{S}$$

Two policies cannot be ordered if each is better on some states. Here $\pi_\uparrow$ is better on 9 tiles and $\pi_\rightarrow$ on 9.

Optimal policy

A finite Markov decision process with $\gamma < 1$ always has an optimal policy π^* with $\pi^* \geq \pi$ for every policy π.

Bellman Optimality Equations

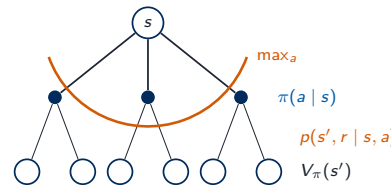
The optimal policy takes the best first action, so the average over actions becomes a maximum, $V_{\pi^*}(s) = \max_a Q_{\pi^*}(s,a)$. For a general policy it is the average $V_\pi(s) = \sum_a \pi(a \mid s)\, Q_\pi(s,a)$.

Bellman optimality equations

$$V_{\pi^*}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_{\pi^*}(s')\big]$$

$$Q_{\pi^*}(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \max_{a'} Q_{\pi^*}(s', a')\big]$$

The maximum makes the equations nonlinear, so a single matrix inversion no longer solves them.



Optimal Action and Policy Improvement

Optimal action

$$a^*(s) = \arg\max_a Q_{\pi^*}(s, a)$$

The optimal policy is the deterministic choice of the best action according to Q_{π^*}. The value functions already contain the whole future, so acting greedily on them is optimal in the long term.

Policy improvement

The greedy policy $\pi'(s) = \arg\max_a Q_\pi(s, a)$ is never worse than π, $V_{\pi'}(s) \geq V_\pi(s)$ on every state. Values can stay equal on some states, and if they stay equal on all of them, π is already optimal.

Policy iteration

Evaluating and improving in turn, the gridworld policy $\pi_\uparrow$ becomes stable after 3 improvements.

Value Iteration

The right-hand side becomes an update,

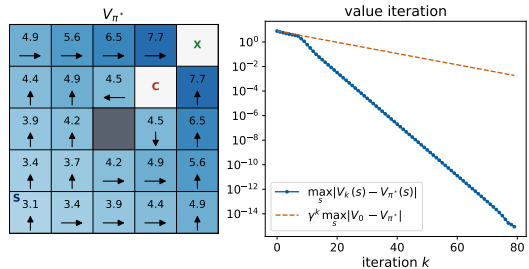
$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_k(s')\big].$$

Each update shrinks the largest error at least by the factor γ,

$$\max_s |V_{k+1} - V_{\pi^*}| \le \gamma \max_s |V_k - V_{\pi^*}|,$$

so $V_k \to V_{\pi^*}$ from any start (a contraction).

The arrows show the optimal policy, which drives around the crater, and $V_{\pi^*}(S) = 3.05$.



5

Learning from Samples



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Model-Based and Model-Free

$$V_{\pi^*}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_{\pi^*}(s') \right]$$

$$Q_{\pi^*}(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} Q_{\pi^*}(s', a') \right]$$

With the dynamics $p(s', r \mid s, a)$ known, or learned as a model and used to plan ahead, the method is **model-based**.

A robot, a lander or a chemical reactor comes without this table, and an agent that learns only from experienced transitions (s, a, r, s') is **model-free**.

Two model-free approaches

1. value-based methods learn V or Q from experience (temporal differences, Q-learning)
2. policy gradient methods learn the policy π_θ directly (REINFORCE)

A Mean Learned from Samples

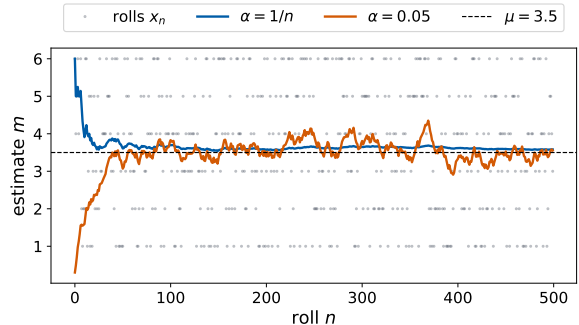
Rolls of a die $x_1, x_2, \ldots$ arrive one at a time, and one estimate m moves a fraction α towards each roll.

Running mean

$$m \leftarrow m + \alpha\,(x_n - m)$$

With $\alpha = 1/n$ this is the exact sample average, with a constant α a moving average. The update needs only samples, never the probabilities.

The Bellman equation is an average over the next step, so it can be learned in the same way.



The Bellman Equation as a Running Mean

The Bellman equation states that the value is a mean over the next step,

$$V_\pi(s) = \mathbb{E}\big[\, r_{t+1} + \gamma\, V_\pi(s_{t+1}) \;\big|\; s_t = s, \pi \,\big].$$

One observed transition (s, a, r, s') gives one sample of the number inside the average, with the unknown $V_\pi(s')$ replaced by the current guess $V(s')$. The running mean of these samples is

Temporal-difference update TD(0)

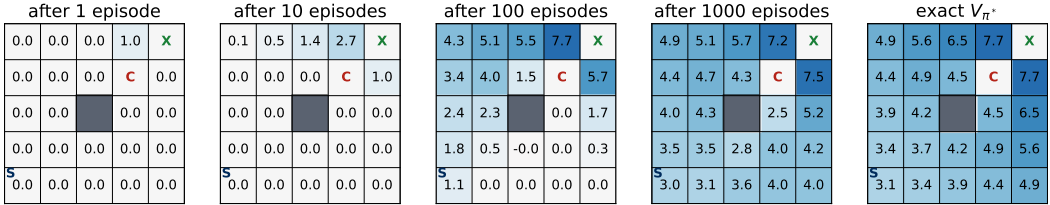
$$V(s) \;\leftarrow\; V(s) + \alpha\big[\, \underbrace{r + \gamma V(s')}_{\text{TD target}} - V(s) \,\big]$$

The **TD error** $\delta = r + \gamma V(s') - V(s)$ compares two predictions of the same return made one step apart, and the later one already contains one real reward.

Averaged over the next step the update is value iteration damped by α, and it converges to V_π if every state keeps being visited and its step sizes decrease slowly enough.

R. S. Sutton, Learning to predict by the methods of temporal differences, Mach. Learn. **3**, 9 (1988)

The Value Spreads Backwards



TD(0) evaluates the optimal policy from $V = 0$ with $\alpha = 0.1$. After one episode only the tile before X has a value. On the next visit the tile before it sees this value in its target $r + \gamma V(s')$, and so the value travels backwards along the path, one step per visit, the recursion of the return applied by learning. Tiles reached only through slips fill in later. After 1000 episodes the map is close to the exact V_{π^*}.

Action Values Instead of State Values

To find the best policy without a model, we learn the Bellman optimality equation as a running mean of samples. The position of the maximum decides which form can be learned in this way.

$$V_{\pi^*}(s) = \max_a \mathbb{E}\big[r + \gamma V_{\pi^*}(s') \mid s, a\big]$$

maximum outside the average

One sample cannot estimate a maximum over averages of different actions, and choosing the action from V_{π^*} needs $p(s' \mid s, a)$ again.

$$Q_{\pi^*}(s, a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q_{\pi^*}(s', a') \;\Big|\; s, a\Big]$$

average outside, over the environment only

The number $r + \gamma \max_{a'} Q(s', a')$ from one transition is a sample of the right-hand side, so the running mean applies.

Q-Learning

Q-learning update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

C. J. C. H. Watkins and P. Dayan, Q-learning, Mach. Learn. **8**, 279 (1992)

ε-greedy behaviour

With probability ε the agent takes a random action, otherwise the action with the largest $Q(s,a)$.

The random actions make sure that every pair (s,a) keeps being updated, which convergence requires.

Off-policy learning

The maximum in the target belongs to the greedy policy, so the agent learns the values of the best policy while it behaves partly at random.

If every pair (s,a) keeps being tried and the step sizes decrease slowly enough, Q converges to Q_{π^*}.

The Q-Learning Loop

The agent knows nothing about the environment. It keeps a table $Q(s, a)$, at first all zeros, and repeats the following.

1. observe the current state s
2. choose an action a, ε-greedy with respect to Q
3. act, and receive the reward r and the next state s' from the environment
4. update one entry of the table towards the Bellman target,
   $Q(s, a) \leftarrow Q(s, a) + \alpha\big[\, r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big]$
5. set $s \leftarrow s'$, and start a new episode when this one ends

Every transition (s, a, r, s') is one sample of the average in the Bellman optimality equation, and the running mean of these samples approaches Q_{π^*}.

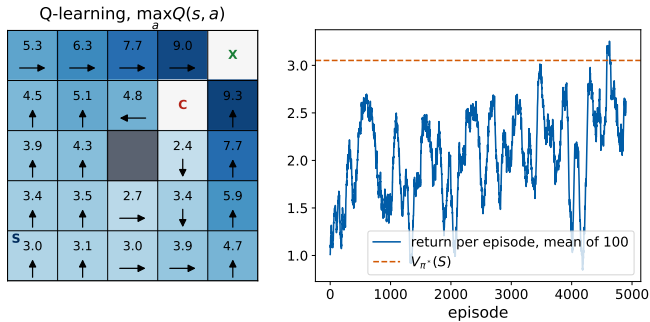
Q-Learning on the Gridworld

5000 episodes, $\varepsilon = 0.1$, $\alpha = 0.1$

The greedy action agrees with π^* on 20 of 22 tiles, and the learned greedy policy has the value 3.03 at S, against the optimal 3.05.

The average return during training stays below $V_{\pi^*}(S)$, since 10% of the actions are random.

Near X some learned values exceed V_{π^*} in this run. A known mechanism contributes, since the maximum over fluctuating estimates tends to pick those that are high by chance.



Exploration and Exploitation

To **exploit** is to use the best strategy known so far, and to **explore** is to try other actions, which may reveal a better strategy.

An agent that only explores performs a brute-force search. An agent that only exploits commits to the first strategy that works, however poor.

The ε-greedy rule mixes the two, and ε is usually lowered during training.



Exploration on a Grid with Two Rewards

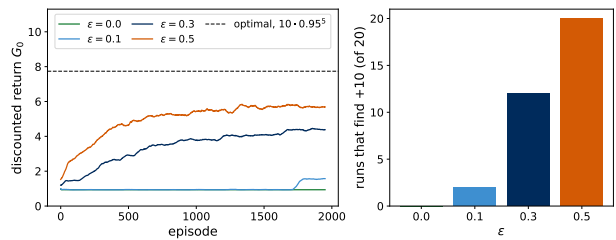
						+10
+1		S				

deterministic 3×7 grid, $\gamma = 0.95$

$+1$ two steps away, worth 0.95

$+10$ six steps away, worth $10 \cdot 0.95^5 \approx 7.7$

A greedy agent finds the $+1$ first and returns there forever.



The final policy walks to $+10$ in 0 of 20 runs with $\varepsilon = 0$ and in 20 of 20 with $\varepsilon = 0.5$.

6

Policy Gradient



JAGIELLONIAN UNIVERSITY
IN KRAKOW

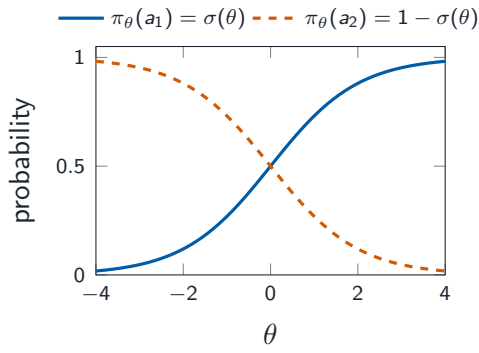
Learning the Policy Directly

Q-learning learns values and acts by $\arg\max_a$. This becomes awkward for continuous actions, such as a throttle or a joint torque, where the maximum is an optimisation problem at every step, and small changes of the values can flip the greedy action abruptly.

The **policy gradient** method writes the policy as a differentiable function $\pi_\theta(a \mid s)$ of parameters θ and moves θ along the gradient of the expected return,

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\big[G(\tau)\big].$$

The action is sampled from π_θ, so the policy explores by itself.



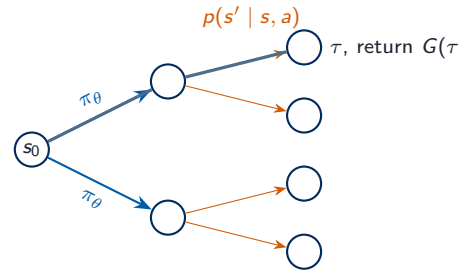
two actions and one parameter, with the logistic function $\sigma(\theta) = 1/(1 + e^{-\theta})$

The Objective as a Sum over Trajectories

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \underbrace{\pi_\theta(a_t \mid s_t)}_{\text{agent}} \underbrace{p(s_{t+1} \mid s_t, a_t)}_{\text{environment}}$$

$$J(\theta) = \sum_\tau p_\theta(\tau)\, G(\tau)$$

The parameters enter only through the choices of the agent. The environment factors are unknown and cannot be differentiated.



Policy Gradient Theorem

The return $G(\tau)$ does not depend on θ, so $\nabla_\theta J = \sum_\tau \nabla_\theta p_\theta(\tau)\, G(\tau)$. Since $\nabla_\theta \ln p_\theta = \nabla_\theta p_\theta / p_\theta$, the log-derivative trick writes $\nabla_\theta p_\theta = p_\theta \nabla_\theta \ln p_\theta$, and

$$\nabla_\theta J = \sum_\tau \underbrace{p_\theta(\tau)}_{\substack{\text{how often} \\ \tau \text{ occurs}}} \underbrace{\nabla_\theta \ln p_\theta(\tau)}_{\substack{\text{direction in } \theta \text{ that} \\ \text{makes } \tau \text{ more likely}}} \underbrace{G(\tau)}_{\substack{\text{how good} \\ \tau \text{ was}}} = \mathbb{E}_{\tau \sim p_\theta}\big[\nabla_\theta \ln p_\theta(\tau)\, G(\tau)\big].$$

The gradient of an average becomes an average, which can be estimated by playing episodes.

R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Mach. Learn. **8**, 229 (1992)

The Dynamics Drop Out

The logarithm turns the product $p_\theta(\tau)$ into a sum,

$$\nabla_\theta \ln p_\theta(\tau) = \nabla_\theta \Big[\underbrace{\ln p(s_0)}_{\text{start, no } \theta} + \underbrace{\sum_t \ln \pi_\theta(a_t \mid s_t)}_{\text{agent, depends on } \theta} + \underbrace{\sum_t \ln p(s_{t+1} \mid s_t, a_t)}_{\text{environment, no } \theta} \Big] = \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t).$$

Rewards collected before a_t do not depend on a_t, so their terms average to zero. What remains of the return is $\sum_{k \geq t} \gamma^k r_{k+1} = \gamma^t G_t$, with G_t the reward-to-go from t.

Policy gradient

exact gradient of J

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \gamma^t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\, G_t \Big]$$

in practice γ^t is dropped, and REINFORCE uses

$$\hat{g} = \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\, G_t$$

Only the policy is differentiated, so the model of the environment is never needed. For $\gamma < 1$ the direction $\hat{g}$ is not exactly the gradient of J (C. Nota and P. S. Thomas, AAMAS 2020, arXiv:1906.07073).

REINFORCE

The policy $\pi_\theta(a \mid s)$ starts from random parameters, and the following is repeated.

1. play N episodes with the current policy, drawing every action from $\pi_\theta(\cdot \mid s_t)$
2. compute for every step the reward-to-go $G_t = r_{t+1} + \gamma r_{t+2} + \dots$
3. take one gradient step on $L(\theta) = -\frac{1}{N} \sum_{\text{episodes}} \sum_t \ln \pi_\theta(a_t \mid s_t)\, G_t$, whose gradient is minus the average of $\hat{g}$ over the episodes, with N the number of episodes

Every sampled action has its log-probability pushed with the weight G_t. A positive G_t makes the sampled action more probable, and a larger G_t pushes harder.

The name REINFORCE refers to this rule, reinforce what worked. The episodes are discarded after one step, because they were played with the old policy.

A Two-Action Example

One state and two actions with fixed rewards $R(a_1) = 2$ and $R(a_2) = 1$. The policy is $\pi_\theta(a_1) = \sigma(\theta) = p$ and $\pi_\theta(a_2) = 1 - p$.

$$J(\theta) = 2p + (1 - p), \qquad \frac{dJ}{d\theta} = p(1 - p)$$

The scores are $\partial_\theta \ln \pi_\theta(a_1) = 1 - p$ and $\partial_\theta \ln \pi_\theta(a_2) = -p$, and one sampled action gives the estimate

$$g = R(a)\, \partial_\theta \ln \pi_\theta(a).$$

At $\theta = 0$, $p = 1/2$, the exact gradient is 0.25.

sampled a	g	θ	$\pi_\theta(a)$
a_1	1	up	rises
a_2	-0.5	down	rises

Both rewards are positive, so every sampled action is pushed up, a_1 more strongly. The mean of g is the exact 0.25, and the single samples scatter with variance 0.5625.

The Baseline

A baseline b is subtracted from the return, $g = (R(a) - b)\,\partial_\theta \ln \pi_\theta(a)$. Since $\sum_a \pi_\theta(a)\,\partial_\theta \ln \pi_\theta(a) = \partial_\theta \sum_a \pi_\theta(a) = 0$, the mean of g is $p(1-p)$ for every b.

baseline	g, action a_1	g, action a_2	mean	variance
$b = 0$	1	-0.5	0.25	0.5625
$b = 1.5$, the mean reward under π	0.25	0.25	0.25	0
$b = 10$	-4	4.5	0.25	18.0625

A baseline near the average return reduces the variance, and a badly chosen one increases it. In an episode the baseline $b(s_t)$ may depend on the state but not on the action. The usual choice is the state value $V_\pi(s_t)$, so that $G_t - V_\pi(s_t)$ estimates the advantage $A_\pi(s_t, a_t)$ and pushes the action up only if it went better than usual from s_t. In practice V_π is a second function V_w, fitted to the observed returns G_t.

Two Routes, Two Averages

value route, Q-learning

learns $Q(s, a)$ and acts greedily

average over *one next step*, the Bellman equation

$Q(s, a) = \mathbb{E}\big[r + \gamma \max_{a'} Q(s', a')\big]$

learned from single transitions (s, a, r, s')

$Q \leftarrow Q + \alpha\,(\text{target} - Q)$

policy route, policy gradient

learns $\pi_\theta(a \mid s)$ directly

average over *whole trajectories*

$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\big[G(\tau)\big]$

its gradient is again an average, estimated from episodes

$\nabla_\theta J = \mathbb{E}\big[\sum_t \gamma^t \nabla_\theta \ln \pi_\theta(a_t|s_t)\, G_t\big]$

in practice γ^t is dropped ($\hat{g}$)

The routes meet in the value function $V(s)$, the baseline of the policy gradient. Actor-critic methods use such a learned V also in place of the full return, by bootstrapping.

The value route needs a maximum over actions and reuses old transitions. The policy route works for any action space, also continuous, with noisier estimates.

7

Deep Networks in Place of Tables

JAGIELLONIAN UNIVERSITY
IN KRAKOW

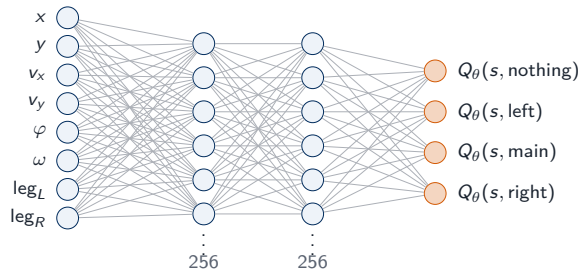
From a Table to a Network

The gridworld needs 22 non-terminal states $\times$ 4 actions, 88 action values in a table.

The state of a lander consists of 8 real numbers that never repeat exactly. No table can hold them, and nearby states should have nearby values.

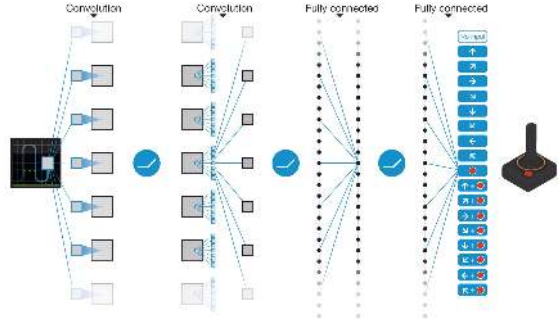
A neural network replaces the table. The network $Q_\theta(s, \cdot)$ takes the state and returns one value per action, and the greedy action is the largest output. In the same way a network $\pi_\theta(\cdot \mid s)$ returns the probabilities of the actions.

V. Mnih *et al.*, Nature **518**, 529 (2015)



Atari Games from Pixels

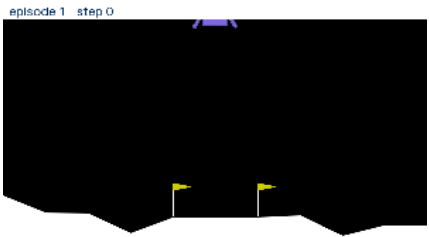




One network architecture learned 49 games. Its input is the last screen frames, its output one value per joystick action, and the reward the change of the score. We use the same idea, a network with one Q value per action, with a small dense network on the state of the Lunar Lander.

V. Mnih *et al.*, Human-level control through deep reinforcement learning, Nature **518**, 529 (2015)

The Lunar Lander



open video
random actions

state	position (x, y), velocity (v_x, v_y), angle φ, angular velocity ω, two leg contacts
action	nothing, left engine, main engine, right engine
reward	more for being closer to the pad and slower, less for tilt, $+10$ per leg on the ground, a cost for fuel, $+100$ for a safe landing and -100 for a crash
episode	ends with a crash, leaving the screen, rest, or after 1000 steps

A return ≥ 200 counts as a landing. The random agent has the mean return -203.3.

Box2D simulation in Gymnasium, M. Towers *et al.*, arXiv:2407.17032

Q-Learning as Regression

The tabular update moves $Q(s, a)$ towards $y = r + \gamma \max_{a'} Q(s', a')$. With a network the same step becomes a loss, minimised by gradient descent over mini-batches B of transitions,

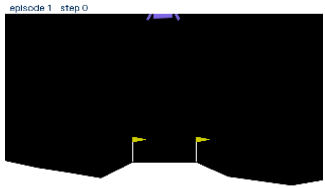
Loss of the deep Q-network (DQN)

$$L(\theta) = \frac{1}{|B|} \sum_{(s,a,r,s') \in B} \ell\big(y - Q_\theta(s, a)\big), \qquad y = r + \gamma \max_{a'} Q_{\theta^-}(s', a'),$$

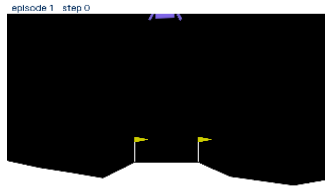
with ℓ a regression loss such as the squared error. The target y plays the role of the label and is held fixed in the gradient. It is $y = r$ when s' is terminal (crash or coming to rest), whereas at the 1000-step time limit the target still bootstraps. The target moves with the network, so the regression can chase itself, and two fixes keep it stable.

- The **target network** Q_{θ^-}, a copy with older weights that follow the trained ones slowly, computes y, so the target changes little from step to step.
- The **replay buffer** stores past transitions, and random mini-batches drawn from it reduce the correlation of consecutive states.

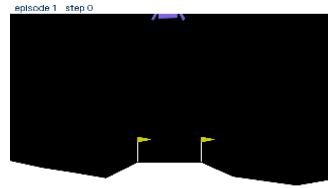
The Lander Learns



open video
random actions



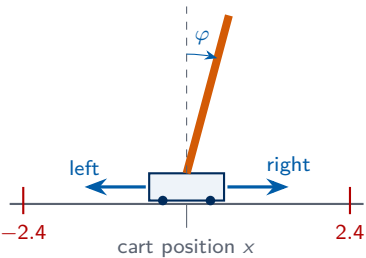
open video
badly trained, 25 000 steps



open video
well trained, 200 000 steps

Each video plays several episodes one after another, with the episode number and its return on top. The random agent tumbles and crashes. The badly trained one has learned to brake and hover, which avoids the crash penalty, but it does not land. The well-trained one descends under control and lands between the flags. On 50 new episodes it reaches the mean return 242.3, above the landing threshold of 200.

The Cart-Pole



the pole must stay within $|\varphi| \leq 12^\circ$ and the cart within $|x| \leq 2.4$

A pole is hinged on a cart, and the agent pushes the cart left or right every 0.02 s.

state	cart position x, velocity $\dot{x}$, pole angle φ, angular velocity $\dot{\varphi}$
action	push left, push right
reward	$+1$ per step with the pole up
episode	ends at $\lvert\varphi\rvert > 12^\circ$, $\lvert x\rvert > 2.4$, or after 500 steps

Random pushes keep the pole up for 23.0 steps on average.

A. G. Barto, R. S. Sutton, C. W. Anderson, IEEE Trans. Syst. Man Cybern. **SMC-13**, 834 (1983)

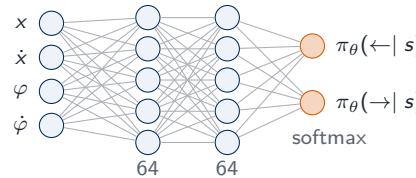
REINFORCE with a Policy Network

The policy network maps the four numbers of the state to the probabilities of the two pushes, and the action is sampled from them.

After every $N = 5$ episodes one gradient step is taken on the REINFORCE loss

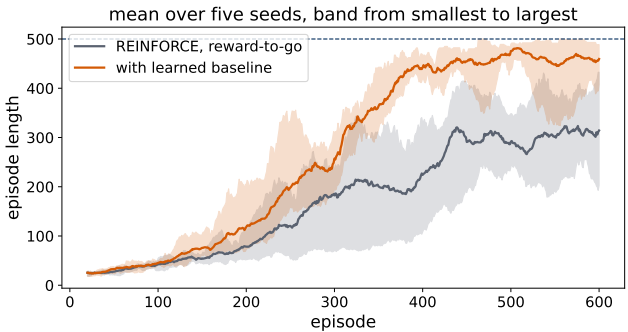
$$L(\theta) = -\frac{1}{N} \sum_{\text{episodes}} \sum_{t} \ln \pi_\theta(a_t \mid s_t)(G_t - V_w(s_t)).$$

The baseline V_w is a second network of the same size, fitted by regression to the observed returns G_t.



policy network $4 \to 64 \to 64 \to 2$, 4610 parameters

Training REINFORCE



600 episodes, one gradient step per five episodes, five seeds each, and the band spans the smallest to the largest seed.

The mean length of the last 50 episodes is 313.3 without and 461.1 with the baseline.

With the baseline the episodes become long sooner and the seeds differ less.

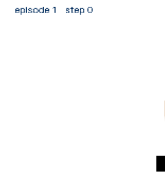
The Pole Is Balanced



open video
random pushes



open video
badly trained, 100 episodes



open video
well trained, 600 episodes

Several episodes play one after another, the random pushes at one fifth of real speed and the trained policies at half speed. With random pushes the pole falls within a second. The badly trained policy keeps it up for a while, until the tilt slowly grows and the pole falls after about 80 steps. The well-trained policy keeps it up for the whole episode of 500 steps with small corrections.

8

Reinforcement Learning in Science



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Discovery as a Game

AlphaZero learns chess, shogi and Go with one algorithm from self-play, with a search guided by a policy and a value network

D. Silver *et al.*, Science **362**, 1140 (2018)

AlphaTensor treats matrix multiplication as a game

AlphaDev found faster sorting routines in assembly code

D. J. Mankowitz *et al.*, Nature **618**, 257 (2023)

AlphaProof learned by reinforcement learning to write proofs in the formal language Lean, and at the 2024 Mathematical Olympiad it solved three of the five non-geometry problems

T. Hubert *et al.*, Nature **651**, 607 (2025)

Search without reinforcement learning

FunSearch and AlphaEvolve let a language model propose programs and keep the best in an evolutionary loop, without a value function or a policy gradient

B. Romera-Paredes *et al.*, Nature **625**, 468 (2023); A. Novikov *et al.*, arXiv:2506.13131 (2025)

AlphaTensor Discovers Algorithms

Multiplying two matrices costs scalar multiplications. Every algorithm for it is a way of writing a fixed three-index tensor as a sum of rank-one pieces, and fewer pieces mean fewer multiplications.

In the **game** the state is the part of the tensor still to be written, an action subtracts one rank-one piece, and every step costs the reward -1. Winning fast means finding a cheaper algorithm.

- trained like AlphaZero, with self-play, a policy and a value network and tree search
- for 4×4 matrices in arithmetic modulo 2 it found 47 multiplications instead of the 49 from Strassen's algorithm of 1969
- thousands of new algorithms for other matrix sizes

a mathematical search turned into a game, solved by reinforcement learning

A. Fawzi *et al.*, Discovering faster matrix multiplication algorithms with reinforcement learning, Nature **610**, 47 (2022)

Treatment Strategies from Hospital Records

In the treatment of sepsis in intensive care a doctor chooses every few hours the doses of intravenous fluids and of drugs that raise blood pressure.

As a **Markov decision process**, the state is the condition of the patient from vital signs and laboratory values, an action is one of 25 dose combinations, and the reward at the end is survival or death.

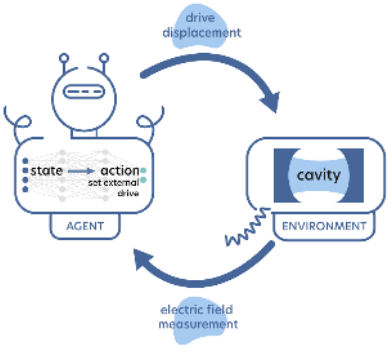
The policy was learned from about 17 000 intensive-care admissions and validated on about 79 000 patients of an independent database, without any new experiment on a patient.

- in the independent database, patients whose actual doses were close to the learned policy had the lowest mortality
- the result is debated, because learning only from logged decisions cannot test what other actions would have done, and the choices of doctors depend on things the records do not show

Here the agent cannot explore, a central difficulty of learning from recorded decisions.

M. Komorowski *et al.*, The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care, Nat. Med. **24**, 1716 (2018)

Feedback Control of a Leaky Cavity



Photons leak out of one mode of a cavity at the rate κ. A weak quantum non-demolition measurement of the photon number gives a noisy signal, and a linear drive of adjustable amplitude acts on the cavity.

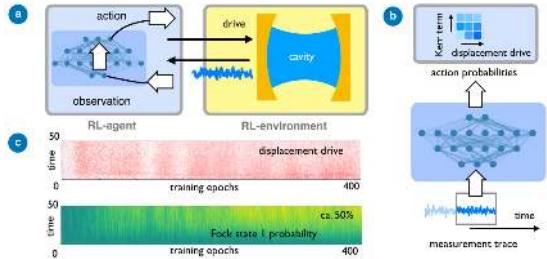
The **task** is to create one photon in the cavity and keep it there, in the Fock state $|1\rangle$.

observation	the last few points of the measured photon-number signal
action	one of N_α amplitudes of the linear drive, set for a time Δt
reward	$r_t = \langle 1|\hat{\rho}_t|1\rangle$, the probability of one photon
algorithm	policy gradient on batches of simulated trajectories

The agent never sees the density matrix $\hat{\rho}_t$ and infers the state from the noisy signal.

F. Marquardt, SciPost Phys. Lect. Notes **29** (2021); figure from A. Dawid *et al.*, arXiv:2204.04198

Learning to Hold One Photon



(b) The network maps the measurement trace to the probabilities of the actions. (c) During 400 training epochs the drive changes and the probability of $|1\rangle$ grows to about 50%. A drive without feedback produces a coherent state $|\alpha\rangle$, whose probability of one photon is $|\alpha|^2 e^{-|\alpha|^2} \leq e^{-1} \approx 0.37$, and the feedback exceeds this benchmark.

In the laboratory a neural network on a programmable chip (FPGA), with a latency below a microsecond, learned from measurements alone to reset a superconducting qubit.

Figure: F. Marquardt, SciPost Phys. Lect. Notes **29** (2021); K. Reuer *et al.*, Nat. Commun. **14**, 7138 (2023)

Methods of This Lecture

method	learns	model needed	example
linear system, value iteration	V_π, V_{π^*} from the Bellman equations	yes	gridworld
temporal differences, TD(0)	V_π in a table, from samples	no	gridworld
Q-learning	Q_{π^*} in a table	no	gridworld
REINFORCE	π_θ by the policy gradient, with a baseline	no	two-action example
deep Q-network	Q_{π^*} as a neural network	no	Lunar Lander
REINFORCE with networks	π_θ and the baseline V_w as neural networks	no	cart-pole

All of them rest on one relation, the value now equals the expected reward plus the discounted value next.

Reading

- R. S. Sutton and A. G. Barto, *Reinforcement Learning. An Introduction*, 2nd ed., MIT Press (2018), incompleteideas.net/book/the-book-2nd.html
- A. Dawid *et al.*, *Modern applications of machine learning in quantum sciences*, arXiv:2204.04198, chapter on reinforcement learning
- M. Bukov and F. Marquardt, *Reinforcement Learning for Quantum Technology*, arXiv:2601.18953 (2026)

Thank You

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



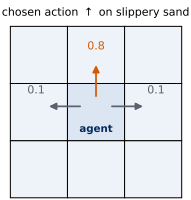


9
Additional Material

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Transition Probabilities on the Gridworld





action $\uparrow$ on a free tile

$p(\text{up} \mid s, \uparrow) = 0.8,$

$p(\text{left} \mid s, \uparrow) = p(\text{right} \mid s, \uparrow) = 0.1$

expected reward, tile below X, action $\uparrow$

$0.8 \rightarrow$ X, reward $+10$

$0.1 \rightarrow$ C, reward -10

$0.1 \rightarrow$ edge, stays, reward 0

$r(s, \uparrow) = 0.8 \cdot 10 + 0.1 \cdot (-10) + 0.1 \cdot 0 = 7$

Convergence to the Bellman Solution

The target contains V itself. Averaged over the random next step it becomes the right-hand side of the Bellman equation, $\mathcal{T}_\pi V$, and the update becomes

$$V \;\leftarrow\; V + \alpha\big(\mathcal{T}_\pi V - V\big) = (1-\alpha)\,V + \alpha\,\mathcal{T}_\pi V.$$

- This is value iteration with damping α. Its errors shrink by the factor $1-\alpha(1-\gamma) < 1$ per update, so it converges, and it can stop only at $V = \mathcal{T}_\pi V$, the Bellman equation.
- The real update changes only the state just visited, and a single sample adds noise of zero mean. With $\alpha_n(s)$ the step size at the n-th visit of s, the estimate converges to V_π if every state is visited infinitely often and $\sum_n \alpha_n(s) = \infty$, $\sum_n \alpha_n(s)^2 < \infty$ in every state.

Temporal-difference learning is value iteration in which the average over the next state is replaced by one sample and a small step.

H. Robbins, S. Monro, Ann. Math. Statist. **22**, 400 (1951), T. Jaakkola *et al.*, Neural Comput. **6**, 1185 (1994)

Monte Carlo and Temporal Difference

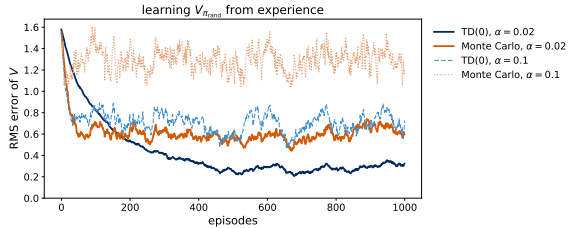
Monte Carlo uses the full return as target,

$$V(s) \leftarrow V(s) + \alpha\big[G_t - V(s)\big]$$

and has to wait for the end of the episode, while G_t collects the randomness of every later step.

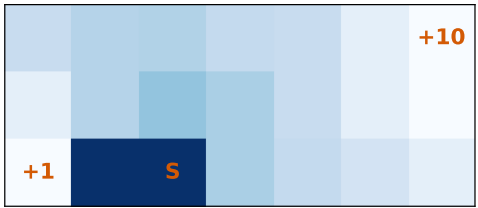
TD(0) uses one reward plus the current estimate, learns after every step and carries the randomness of one step only.

For the random policy on the gridworld and this step size, TD ends with the smaller error. The general reason is that the TD target has a lower variance than the full return.



Where the Agents Go

visits, $\varepsilon = 0.0$ (log scale)



visits, $\varepsilon = 0.5$ (log scale)



The maps count the visits during training on a logarithmic colour scale. The greedy agent never enters the right part of the grid. With $\varepsilon = 0.5$ the agent visits every tile and finds the large reward, at the price of many wasted random moves. Practical agents start with a large ε and lower it during training.

Replay Buffer and Target Network

Replay buffer

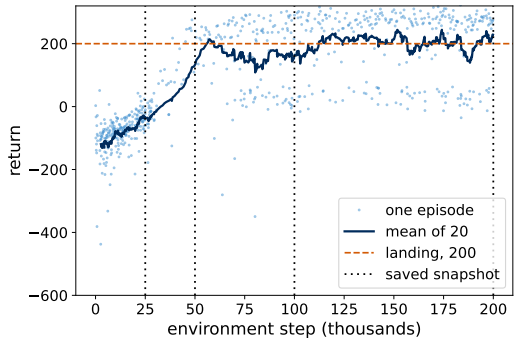
Consecutive states of an episode are almost identical, so mini-batches of consecutive transitions are strongly correlated. The buffer stores the last 100 000 transitions, and training uses random mini-batches of 64 drawn from it, which reuses each transition many times.

Target network

The target y depends on the network being trained, so every step also moves the target. A slow copy $\theta^- \leftarrow (1-\nu)\theta^- + \nu\theta$ with the soft-update rate $\nu = 0.005$ computes y instead.

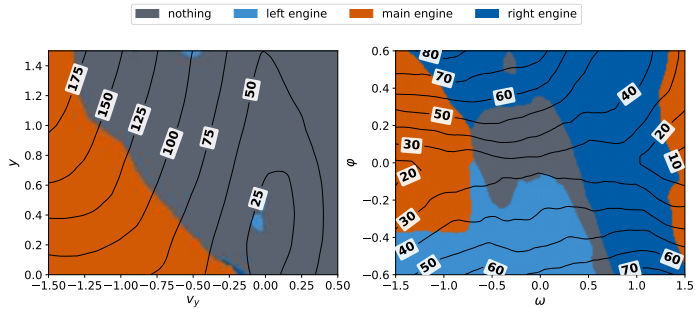
During the first quarter of the training steps ε falls from 1 to 0.05, so the agent explores widely at first and acts mostly greedily later.

Training the Lander



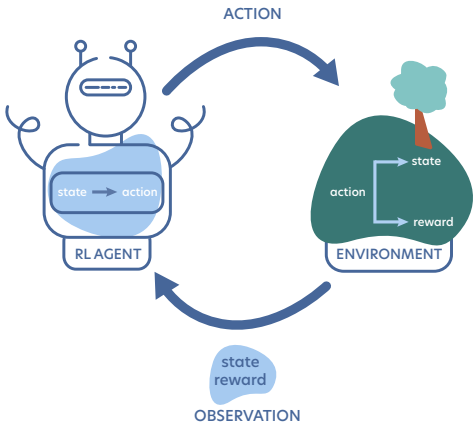
The return of every training episode and its running mean rise from the level of random actions to above the landing threshold of 200. The loss of the regression is a poor guide, since its targets change as the agent improves, and the return is the quantity to watch.

The Learned Control Rule



The colours show the greedy action on two slices of the 8-dimensional state space, and the lines the predicted value. On the left the lander is upright above the pad, with the height y against the vertical velocity v_y, and the main engine fires when the lander falls fast for its height. On the right, at mid height, the angle φ is plotted against the angular velocity ω, and the side engine is chosen by the sign of the tilt, with a slanted boundary. Nobody told the network these rules.

The Agent and Its Environment



The agent observes the state of the environment, chooses an action according to its policy, and receives a new state and a reward.

In supervised learning a student learns from a teacher and is limited by the teacher.

In reinforcement learning the teacher is removed, and the student tries things out and learns from the experience.

A. Dawid *et al.*, arXiv:2204.04198, chapter on reinforcement learning

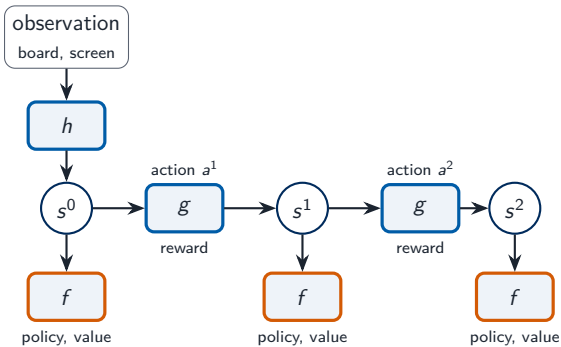
From AlphaGo to MuZero



- AlphaGo (2016): human games, domain knowledge, rules
- AlphaGo Zero (2017): rules only
- AlphaZero (2018): one algorithm for Go, chess and shogi Science **362**, 1140 (2018)
- MuZero (2020): learns a model of the environment itself, also plays Atari Nature **588**, 604 (2020)

figure: deepmind.com

MuZero Plans with a Learned Model



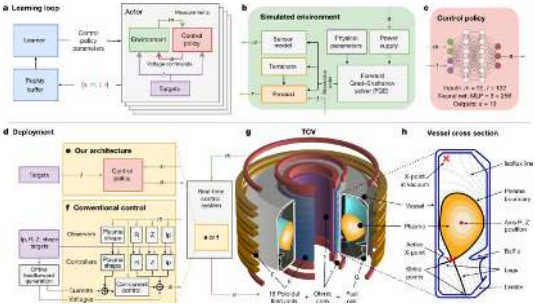
h: observation $\rightarrow$ hidden state
g: hidden state, action $\rightarrow$ next hidden state, reward
f: hidden state $\rightarrow$ policy, value

The model is trained on real games, where along the moves actually played the predicted rewards, values and policies must match the observed ones.

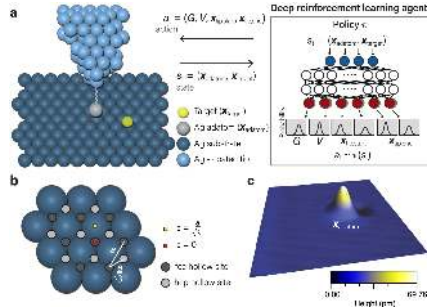
Nothing forces the hidden state to look like a board, and the model learns only what matters for planning. The tree search then runs inside this learned model.

J. Schrittwieser *et al.*, Nature **588**, 604 (2020)

Plasma and Atoms in the Laboratory

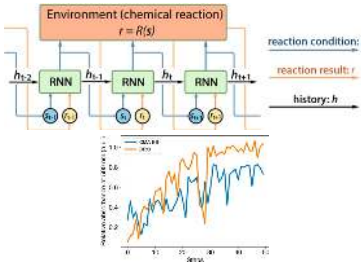


In **fusion plasma** control, a policy network trained only in a simulator of the TCV tokamak (top: learning loop) sets the voltages of the control coils in real time and shapes the plasma (bottom)
J. Degrave *et al.*, Nature **602**, 414 (2022), Fig. 1



Single atoms are moved into designed structures with a scanning tunnelling microscope. The state holds the positions of the silver adatom and of its target, and the action sets the tip bias, the current and the start and end positions of the tip.
I.-J. Chen *et al.*, Nat. Commun. **13**, 7499 (2022), Fig. 1

Chemistry and Beams in the Laboratory



For **chemical reactions** a recurrent network chooses the next reaction conditions from the measured outcomes (orange) and beats a black-box optimiser (blue), with 71% fewer steps reported by the authors
Z. Zhou, X. Li, R. N. Zare, ACS Cent. Sci. **3**, 1337 (2017)

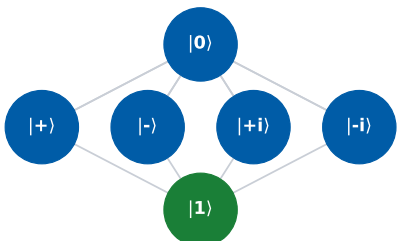
Particle beams

model-free reinforcement learning steers the beam trajectory in the AWAKE electron line and in LINAC4 at CERN
V. Kain *et al.*, Phys. Rev. Accel. Beams **23**, 124801 (2020)

In every case the state is a set of measurements, the action a setting of the apparatus and the reward the quality of the outcome.

A Qubit as a Markov Decision Process

qubit MDP: grey lines are pulses that change the state

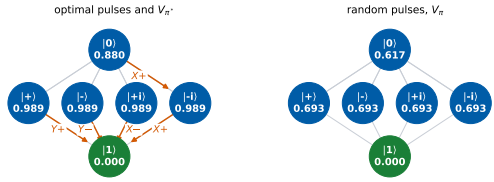


Agent and task

the agent drives a qubit from $|0\rangle$ to $|1\rangle$ with microwave pulses, using as few pulses as possible

$\mathcal{S}$	six points of the Bloch sphere: $\|0\rangle, \|1\rangle$ at the poles, $\|\pm\rangle, \|\pm i\rangle$ on the equator
$\mathcal{A}$	four pulses: 90° rotations about x or y, in either sense
$p(s'\|s,a)$	the rotated state with probability 0.9, while with probability 0.1 the pulse fails and the state stays
r	$+1$ on reaching $\|1\rangle$, which ends the episode
γ	0.9, so fewer pulses are worth more

The Bellman Equation for a Qubit



Optimal pulses, $\gamma = 0.9$. By symmetry all equator states share one value v_e, $|0\rangle$ has v_0, and the terminal $|1\rangle$ has 0.

equator: the right pulse works (0.9) or fails (0.1)

$$v_e = 0.9{\cdot}1 + 0.1\,\gamma\,v_e \;\Rightarrow\; v_e = \frac{0.9}{1-0.1\gamma} = 0.989$$

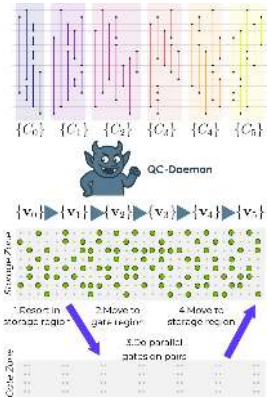
$|0\rangle$: every pulse leads to the equator

$$v_0 = 0.9\,\gamma\,v_e + 0.1\,\gamma\,v_0 \;\Rightarrow\; v_0 = \frac{0.9\,\gamma\,v_e}{1-0.1\gamma} = 0.88$$

Random pulses waste time, with $V(|0\rangle) = 0.617$, $V(\text{equator}) = 0.693$.

The optimal policy is the π pulse made of two 90° pulses. The gridworld needs the same equations for 24 states, without symmetry.

Arranging Atoms as a Game



K. Nakaji *et al.*, arXiv:2506.05536 (2025)

Single atoms held by focused laser beams, the traps, each store one qubit. Gates act on pairs of atoms only in the gate zone, so the atoms needed next move there from storage and later move back. Traps move in whole rows and columns, which cannot cross, and every move adds errors.

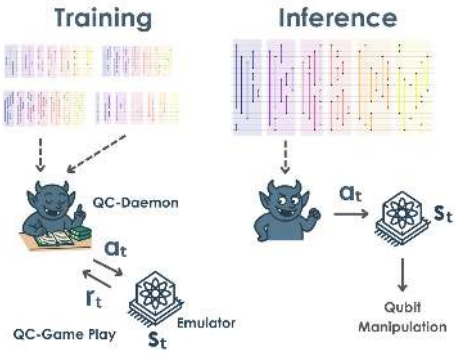
state positions of all atoms, gates still to come

action position of one atom needed soon, one atom after another

reward decrease of the estimated error cost

The gates are fixed. The agent plans where the atoms sit, as if many rovers moved in one gridworld.

Training the Atom Planner



Two transformer networks, one reading the coming gates and one the planned moves, feed separate policy and value heads. They are trained with the policy-gradient method PPO on an emulator of the device.

Training lowers the estimated error cost of the moves. An agent trained on many circuits also lowers the cost of a new circuit it has never seen, whereas an untrained agent raises it.

For a new circuit the trained agent generates a plan without further training, through sequential decisions, one atom and one chunk of the circuit after another.

all results from numerical simulation